ASSIGNMENT 2 · CONTINUATION OF ASSIGNMENT 1

Household object recognition

EfficientNetV2-B0, visual explanations, and a working web demo

Tsoy Ruslan · SE-2420 · Instructor: Nurym Kaiyrkhan · 3 Nov 2026

Abstract

This project upgrades the four-class household object classifier from Assignment 1. It retains the self-collected 847-image dataset and the saved A1 MobileNetV2 model, and tests an ImageNet-pretrained EfficientNetV2-B0 feature extractor with a compact four-class head. A local web app accepts an uploaded image or camera frame, returns probabilities, and overlays a Grad-CAM explanation. The report evaluates both models on the same fixed 127-image test set and includes an augmentation ablation.

A1 RE-EVALUATED	A2 BEST	A2 MACRO-F1
92.91%	99.21%	99.24%
118 / 127 correct	126 / 127 correct	four classes

What was delivered

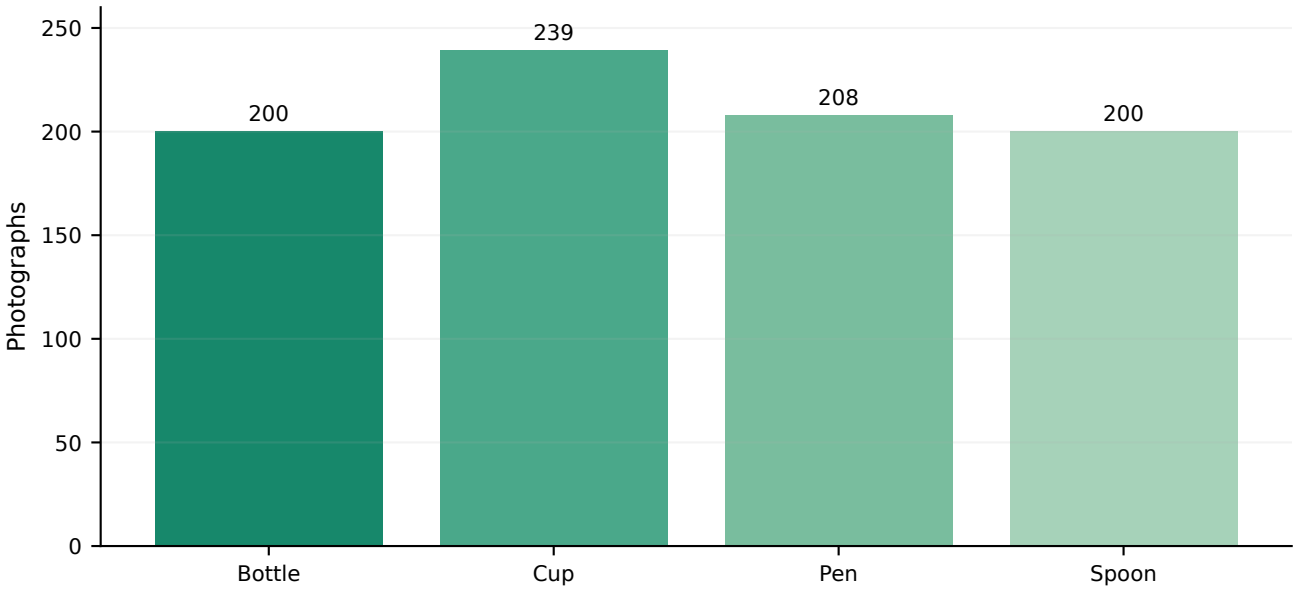
A working local web app, reproducible training and evaluation scripts, frozen model weights, saved classifier heads, a fixed split manifest, training curves, confusion matrices, this PDF report, and an annotated 3 minute 30 second demo video. The original images remain in the local A1 folder; no generated or synthetic images are used.

Main conclusion

EfficientNetV2-B0 improved test accuracy by 6.30 percentage points over the saved A1 model under the fixed A2 protocol. However, photos taken in the same capture sessions can be visually similar, so this small test set does not establish performance on unfamiliar households or lighting conditions. Grad-CAM also exposes cases where contextual pixels influence the model.

The same four-class dataset

The original A1 dataset has 847 real iPhone photographs: bottle (200), cup (239), pen (208), and spoon (200). No images were generated, and the synthetic share is 0%. The source photographs include EXIF metadata; the app re-encodes sample images without metadata and does not store user uploads.



Fixed, disjoint evaluation split

Subset	Images	Share	Role
Training	593	70.0%	Fit classifier
Validation	127	15.0%	Early stopping
Test	127	15.0%	Final metrics

The A1 Keras directory split with seed 42 was reconstructed for training versus holdout. Its holdout was divided once with a seeded shuffle and saved in artifacts/split.json. The A1 code previously took samples from a reshuffled dataset stream, so the original reported 95.28% test accuracy is historical and not directly comparable with the fixed A2 test. We re-evaluated its saved model on this exact A2 test set instead.

EfficientNetV2-B0 transfer learning

EfficientNetV2 is a later architecture from the EfficientNet family, designed with training-aware architecture search and Fused-MBConv blocks for improved training efficiency and parameter use [1]. Neurohive explains the EfficientNet family’s efficiency-oriented scaling [2]. This project uses the small B0 implementation supplied by Keras with ImageNet weights [3], suitable for a CPU-only local demo. We do not claim it is the globally best model in 2026.



Why it helps this topic

The dataset is small and contains varied surfaces, backgrounds and object shapes. Frozen ImageNet features supply general visual patterns learned on a much larger dataset. Only the compact classifier head is fitted here, which limits the number of parameters that 593 training images must estimate. Keras EfficientNetV2 includes its input rescaling inside the base model [3].

Training and explanation

The frozen base produces a spatial feature map for each image. Global average pooling produces a feature vector. A 64-unit ReLU layer, dropout (0.25), and four-way softmax are trained with Adam (learning rate 0.001), batch size 32, up to 35 epochs, and early stopping on validation loss (patience 7). Grad-CAM weights the final feature maps using gradients of the selected class score and overlays the positive response on the input image [4].

Task-aligned metrics

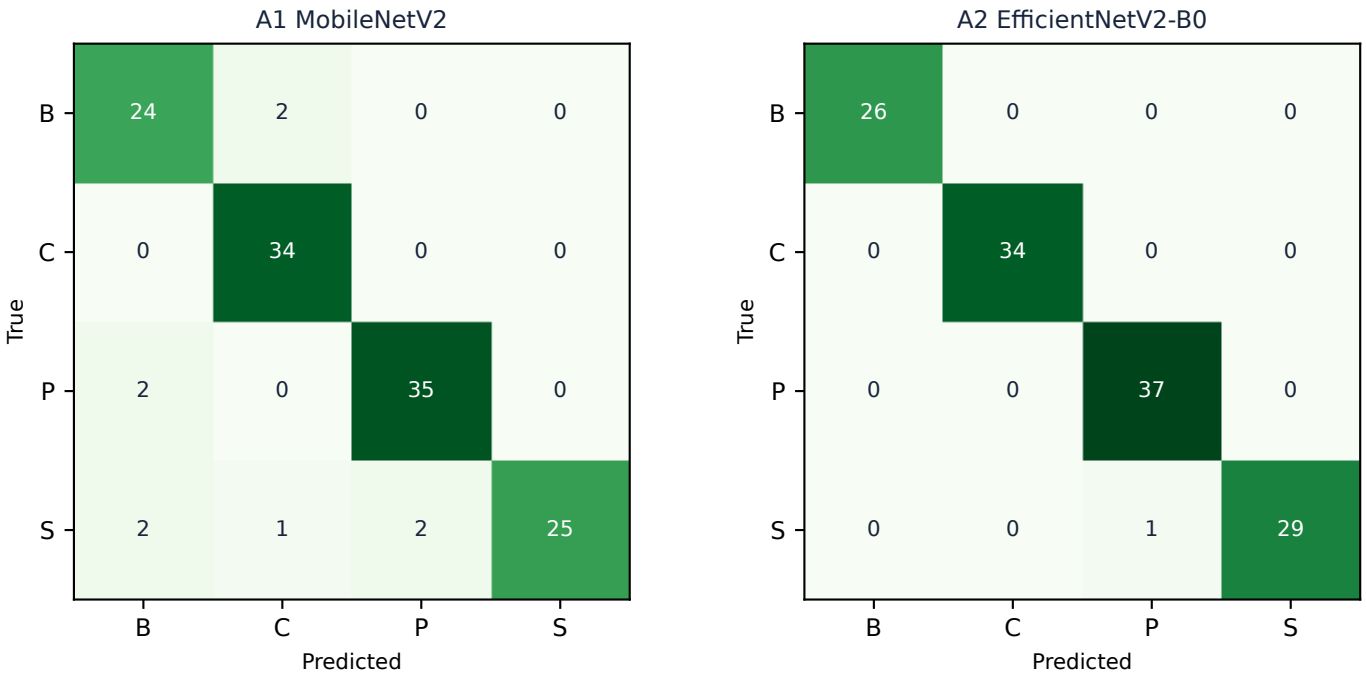
This is whole-image classification, so accuracy and macro-F1 are used. mAP@0.5 and IoU require detection boxes or segmentation masks; neither is annotated in this dataset. Inference latency is measured on the local CPU and includes Grad-CAM generation.

CONTROLLED COMPARISON

A2 results on one fixed test set

Model	Accuracy	Macro-F1	Correct
A1 MobileNetV2 (saved)	92.91%	92.54%	118/127
A2 EfficientNetV2-B0	99.21%	99.24%	126/127
A2 + augmentation	97.64%	97.53%	124/127

All rows use the same 127 physical test images and four classes. A1 is the saved fine-tuned model from Assignment 1, re-evaluated here. The strongest A2 run gains 6.30 percentage points of accuracy and 6.70 points of macro-F1. Its sole error is a spoon predicted as a pen. With 127 related photos, these numbers are descriptive rather than a claim of general performance.

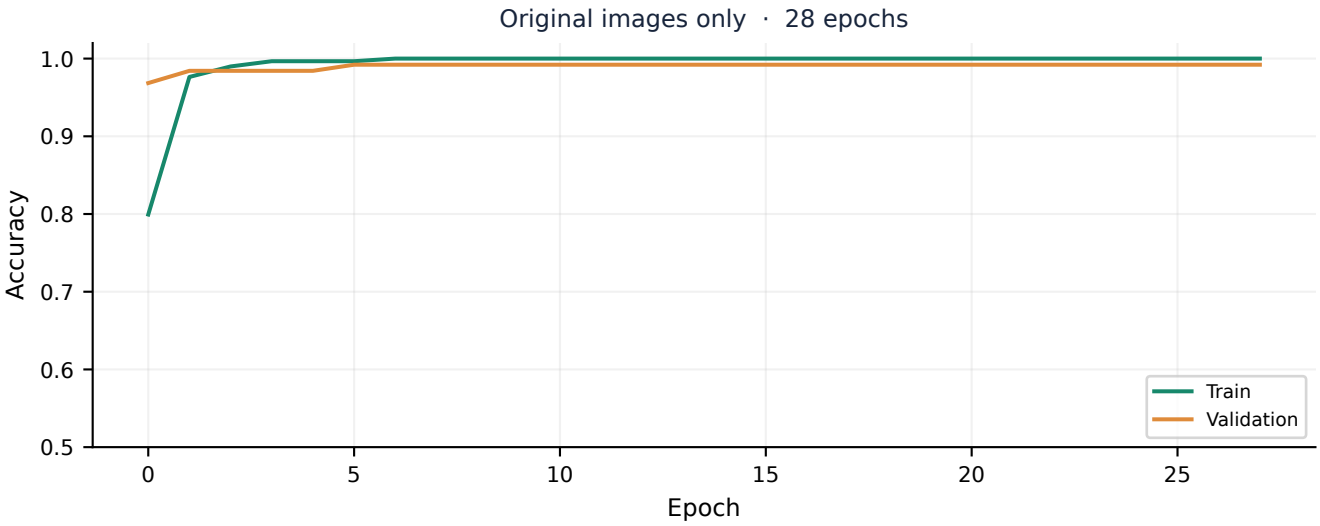


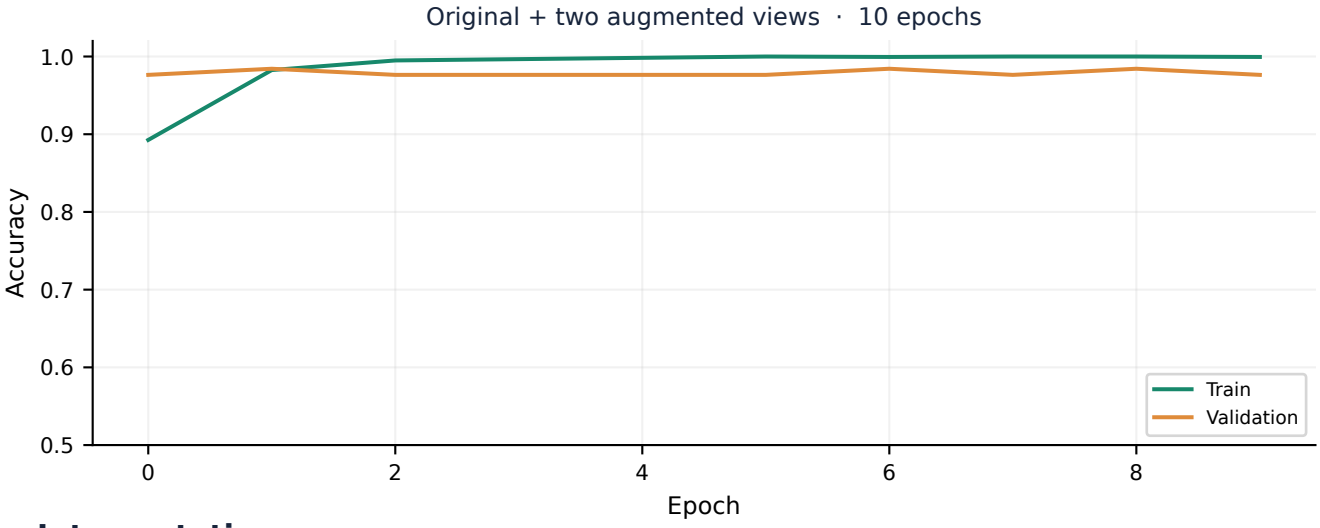
B = bottle, C = cup, P = pen, S = spoon. Rows are true labels; columns are predictions.

ABLATION EXPERIMENT

What did augmentation change?

The frozen EfficientNetV2 features and classifier design were held constant. The ablation changed only the head’s training data: original photos alone versus original plus two independent views generated with horizontal flip, rotation, zoom and translation. Validation and test photos were never augmented. The curves below come from actual Keras training histories.





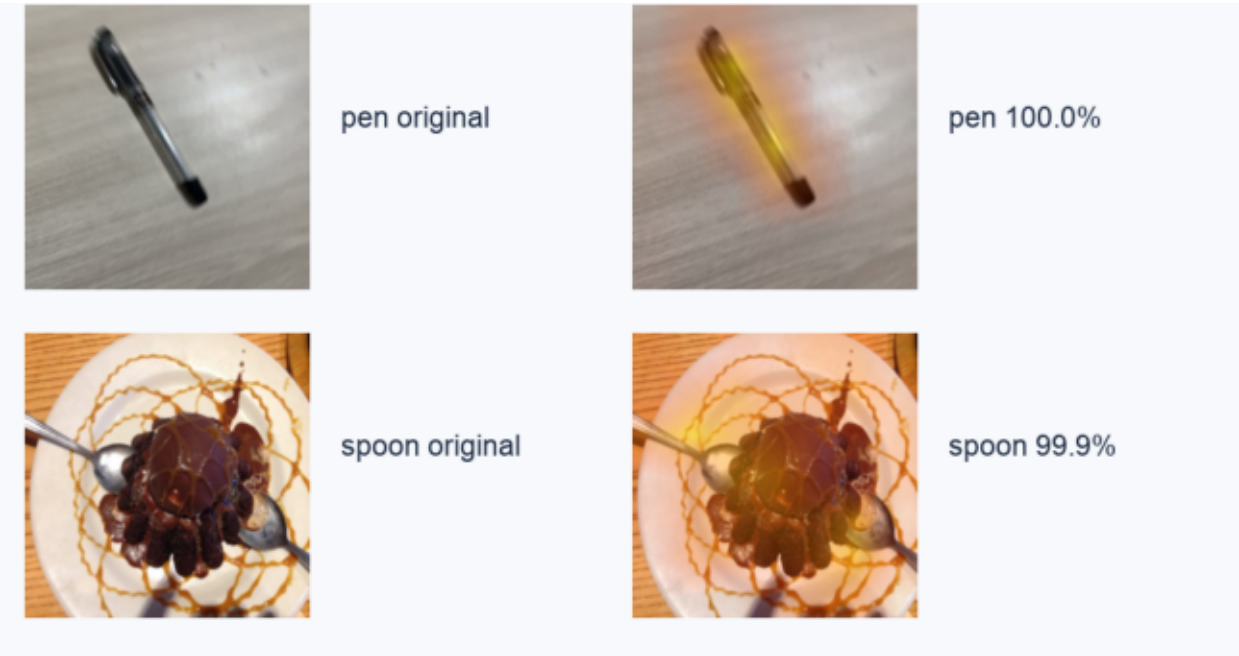
Interpretation

Augmentation reduced accuracy from 99.21% to 97.64% and macro-F1 from 99.24% to 97.53%. This difference is two test images; the experiment cannot establish that augmentation is generally harmful. Strong transformations may change useful details on small objects. The no-augmentation run is used in the app because it scored best on this test. Hyperparameter choice still used the validation set.

Prediction with a visual explanation

The local web app supports image upload, drag-and-drop, camera capture and four example images. It shows the predicted label, all four class probabilities, a Grad-CAM overlay, per-image latency, a rolling average over 30 requests, and the device/browser environment. The overlay is class-specific but coarse: it is not an object mask or proof of causal reasoning.

Figure 1. Pen and spoon examples: original (left), heatmap (right).



What the examples show

On the pen, activation follows the object. On the spoon example, activation also covers the dessert and plate; this warns that context can influence the decision. The four re-encoded example images took approximately 0.18–0.23 seconds each on the local CPU, including heatmap computation. These are illustrative timings, not a cross-device benchmark.

How to run and interpret the project

Consent, privacy and synthetic media

The dataset consists of self-collected household-object photos, not generated media. No people or faces are intentionally used. Original iPhone files can contain location EXIF; keep the raw dataset local and obtain permission before publishing any recognizable person, home, or location. Demo sample images are re-encoded without metadata. Browser uploads are processed locally and not retained by the server. Synthetic share: 0%; no watermark is needed because no synthetic images are displayed.

Limitations and possible misuse

The test contains only 127 images and may share capture conditions with training. A same-class perceptual-hash check found 16/127 test photos within Hamming distance 4 of a training photo, suggesting visual similarity across the split. This can inflate apparent generalization. Background cues, domain shift, unknown objects (e.g., a key), confidence calibration, and the coarse nature of Grad-CAM remain concerns. The app always selects one of four labels; low confidence should prompt human review.

Reproduce

Environment: macOS, Python 3.13, TensorFlow 2.21, Keras 3.15, scikit-learn, NumPy, Pillow and Matplotlib. Put the A1 dataset in assgnment 1 cv/dataset. Run the Python in assgnment 1 cv/.venv: python train.py to rebuild the split, feature vectors, heads, curves and metrics. Run python server.py, then open http://127.0.0.1:8000. Pretrained B0 weights are in weights/; the chosen four-class head is artifacts/no_augmentation.keras. Input: JPEG/PNG/WebP image, maximum 10 MB. See README.md for exact commands.

References

[1] Tan & Le (2021), EfficientNetV2: Smaller Models and Faster Training. https://arxiv.org/abs/2104.00298

[2] Neurohive (2019), EfficientNet: Achieving State-of-The-Art Results. https://neurohive.io/en/news/efficientnet-achieving-state-of-the-art-results-by-systematically-scaling-deep-neural-networks/

[3] Keras, EfficientNetV2 API and preprocessing. https://keras.io/api/applications/efficientnet_v2/

[4] Selvaraju et al. (2017), Grad-CAM. https://openaccess.thecvf.com/content_ICCV_2017/papers/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.pdf